SEMANA 10 · UNIDAD 2

Cierre de Unidad 2: pipeline CI/CD genérico con GitHub Actions

Métricas DORA, etapas del pipeline, federación OIDC, GitOps y estrategias de publicación

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) · Ingeniería de Sistemas

Integración de Soluciones para Plataformas Cloud · Periodo 2026-2 · Barranquilla, Colombia

PARTE 1 DE 4

01

Métricas de DORA: qué mide un buen pipeline

Cinco métricas de desempeño de entrega, cómo usarlas bien y cuándo se convierten en trampa.

PARTE 1
Métricas de DORA: qué mide un buen pipeline

PARTE 2
Etapas del pipeline: construir, probar, escanear, publicar

PARTE 3
GitHub Actions: workflows, secretos y ambientes

PARTE 4
Despliegue: push, GitOps y estrategias de publicación

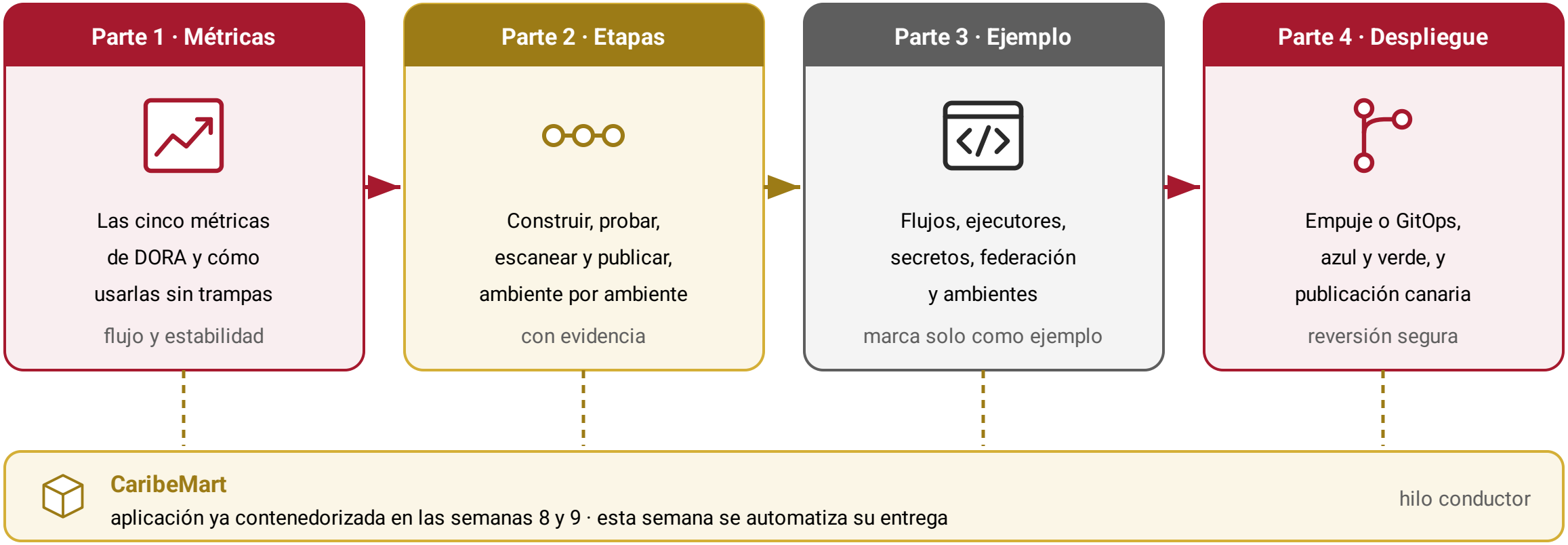
CUC UNIVERSIDAD DE LA COSTA

MAPA DE LA SESIÓN

Cuatro bloques para cerrar la Unidad 2

Primero medimos, después construimos el pipeline, luego lo ejemplificamos y cerramos con el despliegue.

Semana 10 · cierre de la Unidad 2 · un video por parte

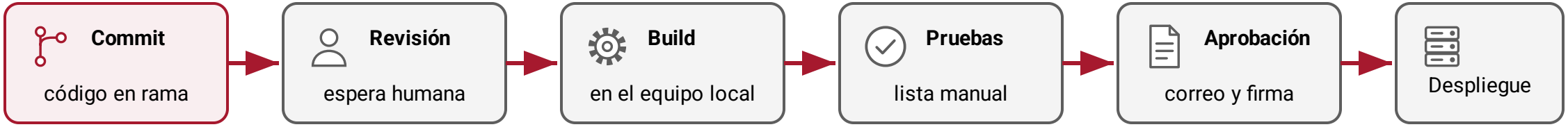


Elaboración propia · Hilo conductor: CaribeMart (caso del curso)

CaribeMart publica a mano y el negocio no espera

Cada paso manual es una espera: revisión, construcción, pruebas y despliegue fuera del control del equipo.

Publicación manual de hoy



seis traspasos, cinco esperas, ninguna evidencia automática

El costo oculto

Cada espera acumula riesgo: el cambio envejece, la rama crece y el error se descubre tarde.

Y nadie responde con evidencia qué versión está corriendo ni quién la aprobó.

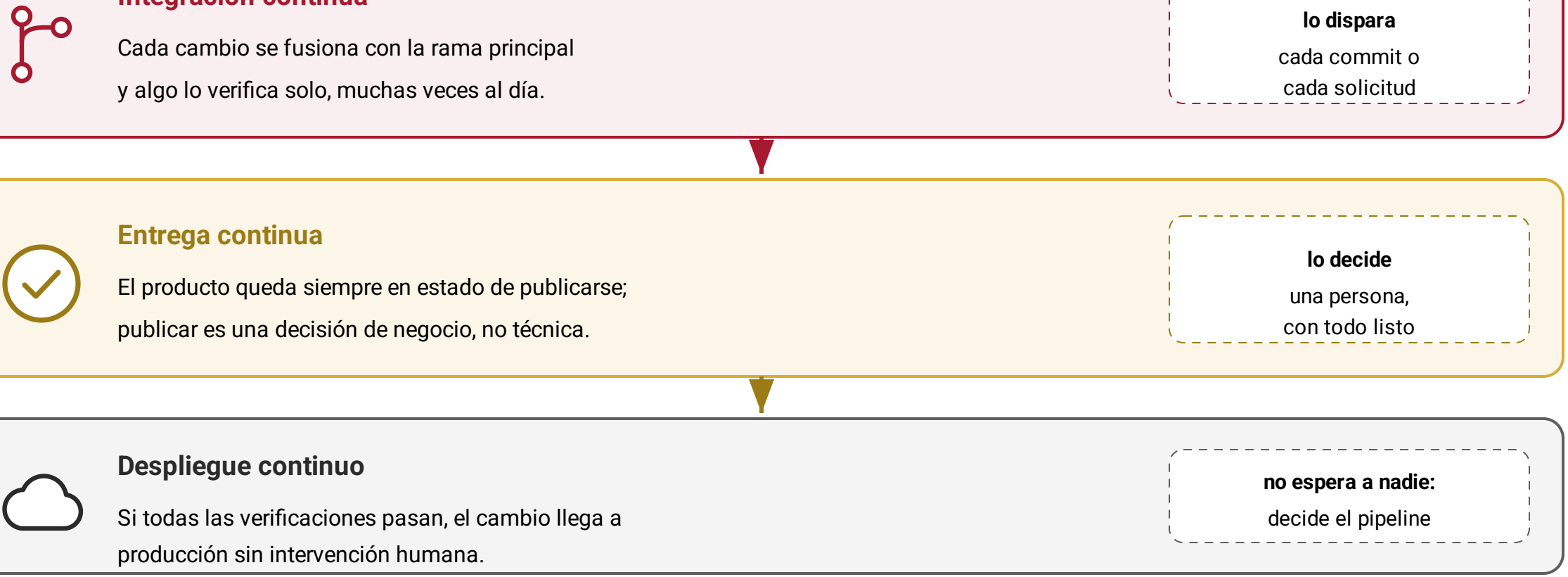
La meta de esta semana

Un solo disparo: el commit activa una cadena automática que construye, prueba, escanea y publica, y deja registro de todo lo que pasó en cada paso.

Caso del curso: CaribeMart (proyecto de aula, ficticio) · Diagrama: elaboración propia

Integración continua, entrega y despliegue continuo

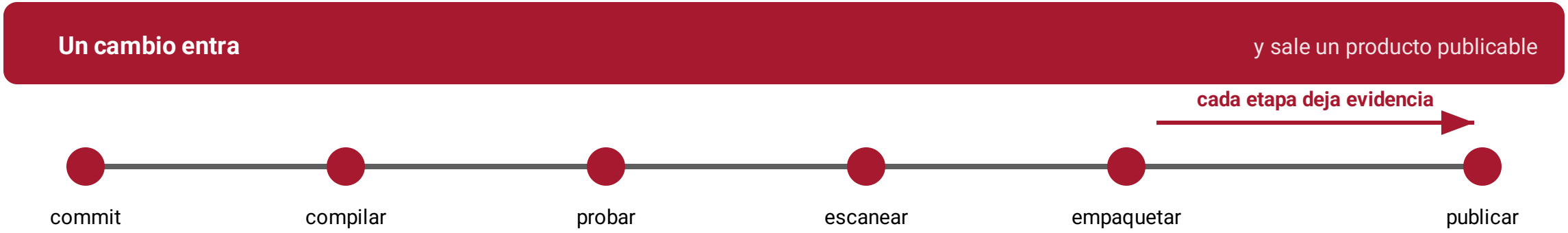
Tres palabras que se usan como sinónimos y no lo son: cambian quién decide y cuándo.



Basado en: Fowler, Continuous Integration [4] · Humble y Farley, Continuous Delivery [3] · Diagrama: elaboración propia

El pipeline: visible, con retroalimentación y a demanda

Tres propósitos del pipeline de despliegue según Continuous Delivery.



Visibilidad

Todo el proceso se ve en un solo lugar: quién hizo qué, con qué resultado y cuánto tardó.

nadie pregunta al aire

Retroalimentación

El fallo aparece minutos después del cambio, con el recuerdo del autor todavía fresco.

error barato, no error caro

A demanda

Cualquier versión probada se lleva a cualquier ambiente con un botón, sin armar nada a mano.

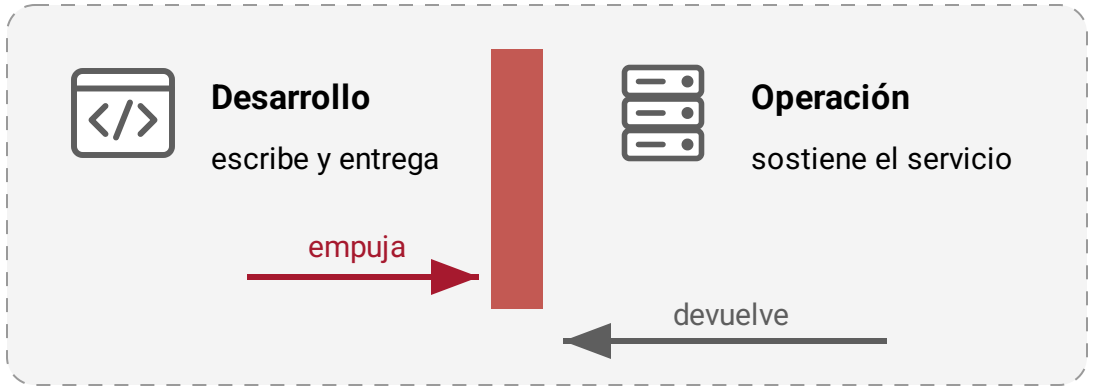
la reversión usa el mismo camino

Basado en: Humble y Farley, Continuous Delivery, capítulo 10 [3] · Diagrama: elaboración propia

DevOps no es un puesto: es responsabilidad compartida

Antes: desarrollo empuja y operación recibe. Después: un solo equipo con las mismas métricas.

Antes · dos equipos y un muro



la culpa viaja de ida y vuelta

Después · un equipo con un ciclo



la responsabilidad del servicio es una sola

Lo que une a los dos escenarios es el dato: las mismas métricas para los dos lados

Métricas de flujo

Frecuencia de despliegue y tiempo de cambio: las comparten los dos lados.

Métricas de estabilidad

Si la tasa de fallo sube, el problema es del sistema, no de una persona.

Basado en: DORA, métricas de desempeño de entrega [1] · Diagrama: elaboración propia

Cinco métricas de DORA: flujo e inestabilidad

Tres de flujo y dos de inestabilidad: así las agrupa la fuente oficial.

FLUJO · cuánto pasa y qué tan rápido

Frecuencia de despliegue

¿cuántas veces llega a producción por día?

Tiempo de cambio

del commit al cliente, sin esperas dormidas

Tiempo de recuperación

cuánto tarda el equipo en sanar un despliegue

la recuperación también es flujo: mide la rapidez de reacción, no la solidez

INESTABILIDAD · qué tan sólido es

Tasa de fallo al desplegar

¿qué porcentaje exige arreglo o reversión?

Tasa de reaparición de trabajo

¿cuántos cambios se rehacen poco después de publicarse?



Cómo se leen juntas

La investigación de DORA lo repite: velocidad y estabilidad no son un intercambio; los mejores van bien en las cinco a la vez.

Si entregas mucho pero rehaces mucho, no estás entregando bien.

flujo sin solidez es riesgo corrido

mide el sistema, no a las personas

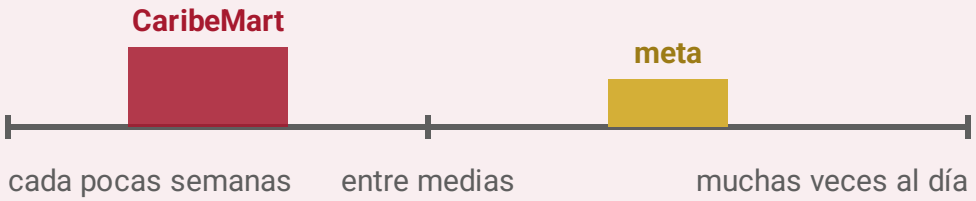
Fuente: DORA, guía oficial de métricas de entrega [1] · Diagrama: elaboración propia

Frecuencia de despliegue y tiempo de cambio

Dos métricas que dependen del pipeline, no del ánimo del equipo.

Frecuencia de despliegue

cuántas veces llega a producción por día



Sube solo si construir y publicar es barato y repetible. Subirla a la fuerza, sin pipeline, solo produce cambios más grandes.

ojo: subir la meta sin pipeline solo acelera los cambios grandes, no el flujo

Tiempo de cambio

del commit al cliente, sin espera dormida

Commit listo para → *ese lapso mide la métrica* → **Cliente** en uso

lo que no se ve también cuentan
las horas en cola de aprobación

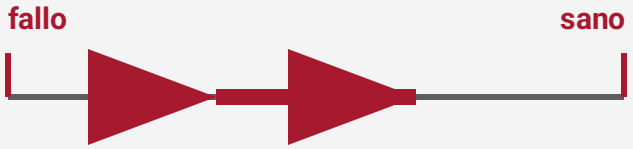
Se mide sobre el reloj real del cambio, no sobre el tiempo de ejecución del equipo. Ahí aparece la cola de aprobación y la espera por el ambiente de pruebas.

Fuente: DORA, guía oficial de métricas de entrega [1] · Caso: CaribeMart (ficticio) · Diagrama: elaboración propia

Recuperación, fallo al desplegar y trabajo repetido

La recuperación mide la rapidez de reacción; el fallo y la reaparición, la solidez de lo publicado.

Tiempo de recuperación



arreglar o revertir · DORA la cuenta en el flujo

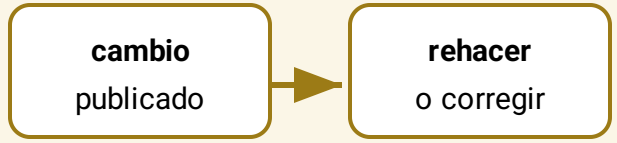
No cuenta el error en sí: cuenta cuánto se tarda el equipo en volver a estar operativo.

Tasa de fallo al desplegar



Un cambio entra aquí si exige un arreglo o una reversión después de publicarse.

Tasa de reaparición



Cuenta los cambios que se rehacen poco después de publicarse.

Leídas juntas

Si despliegas seguido pero cae todo, la velocidad es de mentira. Si nada cae porque no se publica nada, la calma también. Las cinco van de la mano.

Fuente: DORA, guía oficial de métricas de entrega [1] · Caso: CaribeMart (ficticio) · Diagrama: elaboración propia

LECTURA CRUZADA

Velocidad y estabilidad se mejoran a la vez

Los cuatro escenarios posibles y por qué la meta es el cuadrante superior derecho.



Qué dice la fuente oficial

Mejorar la velocidad y la estabilidad a la vez está asociado a un mejor desempeño de entrega y a un mejor desempeño organizacional.

DORA · métricas de desempeño de entrega

Cómo usarlas sin hacer daño

- Compárense con ellos mismos, en el tiempo
- Discutan el sistema, no a la persona
- Revisen las cinco juntas, nunca una sola

Basado en: DORA, guía oficial de métricas de entrega [1] · Diagrama: elaboración propia

CUIDADO

Cuatro trampas al usar las métricas

Los errores típicos al convertir los números en objetivos del equipo.

1 · El promedio esconde la cola

una media mensual tapa los picos

Si el promedio es bonito pero cinco cambios tardaron semanas, la cola sigue ahí.

miren la mediana y el peor caso

2 · La meta individual la corrompe

medir personas rompe el sistema

Si el bono depende de la métrica, aparecen los atajos y las cifras se vuelven inútiles.

se mide el proceso, no a la gente

3 · Comparar lo incomparable

contextos distintos, cifras distintas

Un equipo de mantenimiento y uno de producto no se leen igual en la misma tabla.

compare cada equipo consigo mismo

4 · Medir solo lo fácil

lo que se mide es lo que se cuida

Si solo contamos ejecuciones, nadie mira la espera entre el commit y la ejecución.

incluyan el tiempo en cola

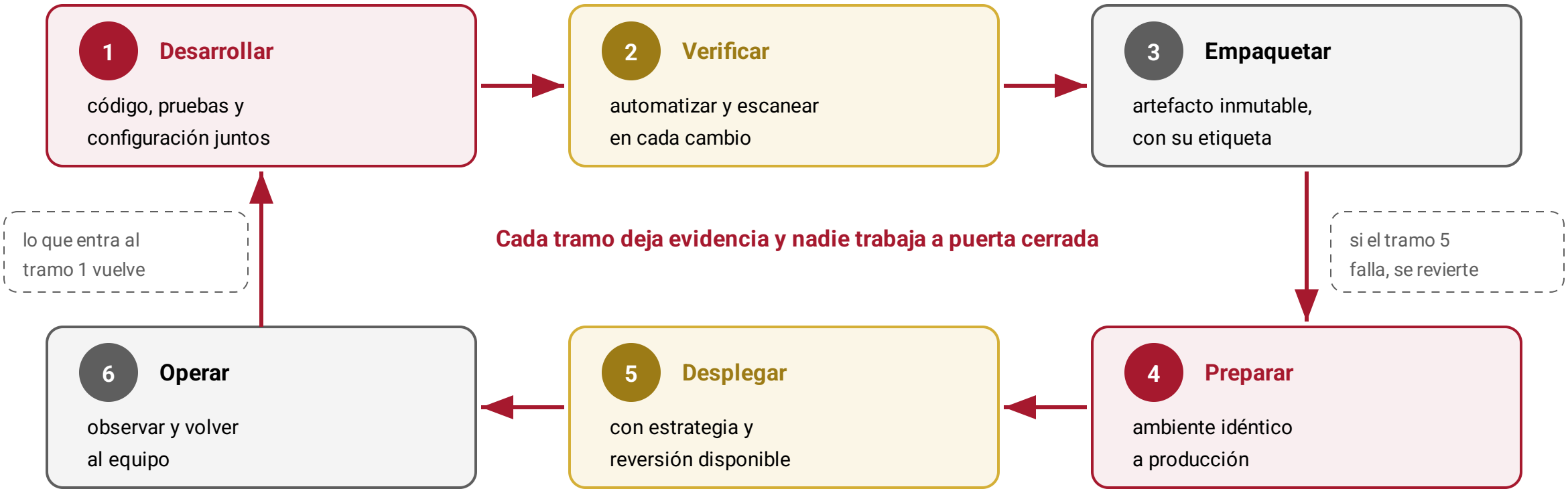
regla: la métrica abre la conversación, nunca la cierra

Basado en: DORA, guía oficial de métricas de entrega [1] · Diagrama: elaboración propia

PROCESO

Seis tramos que el pipeline debe cubrir

El recorrido de un cambio desde el equipo hasta el usuario y de regreso.



Basado en: Humble y Farley, Continuous Delivery, capítulo 10 [3] · Diagrama: elaboración propia

Tu línea base: tres datos, sin herramienta todavía

Antes de automatizar hay que saber dónde se está parado.

Tres pasos esta semana

1. **Anota el último despliegue exitoso**
   fecha, hora y quién lo hizo

2. **Cuenta los despliegues de la semana**
   incluidos los manuales y los de emergencia

3. **Marca cuántos exigieron arreglo**
   o reversión después de publicarse

sin hoja de cálculo ni tablero:
basta un registro escrito por semana

Registro mínimo de la semana

fecha	despliegues · arreglos · espera
semana pasada	
esta semana	
próxima semana	

Con tres columnas aparece la tendencia: si suben los despliegues y bajan los arreglos, el sistema está mejorando.

la tendencia vale más que el dato aislado

Elaboración propia · método de línea base alineado con DORA [1]

MINI-RETO

Pausa el video: ¿dónde se rompe tu cadena?

PAUSA Y RESPONDE

CaribeMart pasó de 1 despliegue por semana a 8, pero las reversiones pasaron de 1 a 6. Con las cinco métricas de DORA, ¿qué le estás diciendo al equipo con ese patrón?

Pista: separa el **flujo** de la **inestabilidad** y mira qué gráfico subió junto con qué otro.

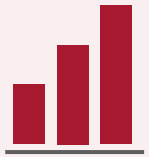
Escribe una frase: si el sistema no mejora, ninguna métrica individual salvará la publicación.



8 despliegues

por semana

antes: 1





6 reversiones

en la semana

antes: 1

Ejercicio de elaboración propia · caso CaribeMart (ficticio)

RESUMEN

Cinco ideas para llevarte de esta parte

Lo que sostiene todo lo que sigue: primero medir, después construir.

1 · Medir primero

Sin línea base no hay mejora demostrable: solo impresiones.

tres datos alcanzan

2 · Cinco métricas

Frecuencia, tiempo de cambio y recuperación son flujo; fallo y reaparición, inestabilidad.

3 · Las dos a la vez

Velocidad sin solidez es ruido; solidez sin velocidad es estancamiento.

se miran juntas

4 · Medir el sistema

La métrica abre la conversación y nunca se vuelve meta individual.



5 · El pipeline es el que da el dato

Cada paso de la cadena deja evidencia automática. Por eso, antes de discutir umbrales, hay que construir la cadena: eso es exactamente lo que sigue.

Siguiente: etapas del pipeline

parte 2 · construir, probar, escanear, publicar

PARTE 2 DE 4

02

Etapas del pipeline: construir, probar, escanear, publicar

Las cuatro etapas, los ambientes con aprobación y la trazabilidad de cada artefacto.

PARTE 1
Métricas de DORA: qué mide un buen pipeline

PARTE 2
Etapas del pipeline: construir, probar, escanear, publicar

PARTE 3
GitHub Actions: workflows, secretos y ambientes

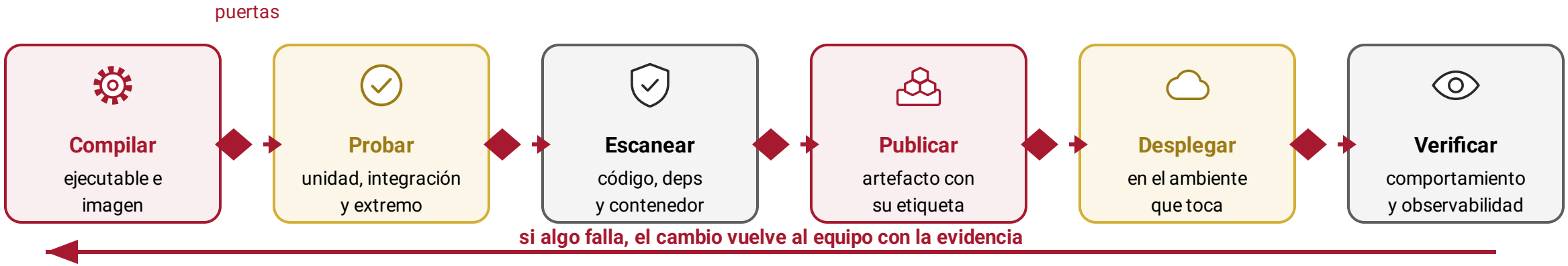
PARTE 4
Despliegue: push, GitOps y estrategias de publicación

CUC UNIVERSIDAD DE LA COSTA

Seis etapas, cinco puertas, cero decisiones a mano

El flujo de un cambio desde el equipo hasta el ambiente, con una puerta de calidad entre etapa y etapa.

cada etapa deja evidencia y cada puerta decide sola



Si una puerta falla

la cadena se detiene ahí

No avanza nada: ni etiqueta, ni despliegue, ni cambio en el ambiente. El equipo recibe el detalle del fallo con la misma ejecución.

Si todas pasan

el mismo artefacto sigue de camino

Lo que se probó en la tercera puerta es lo que llega al ambiente final: nadie reconstruye entre etapa y etapa.

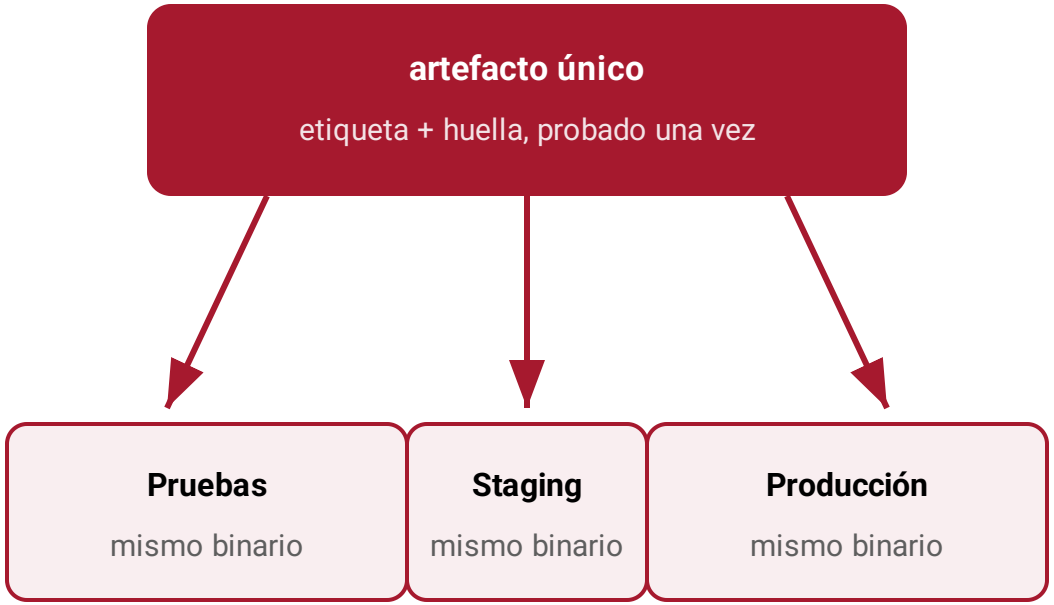
Basado en: Humble y Farley, Continuous Delivery, capítulo 10 [3] · Diagrama: elaboración propia

REGLA 1

Compilar una vez y publicar en todas partes

Cada reconstrucción crea un binario distinto, y lo distinto no está probado.

Correcto: un artefacto, tres destinos



Incorrecto: compilar en cada ambiente



tres compilaciones, tres resultados posibles

El efecto

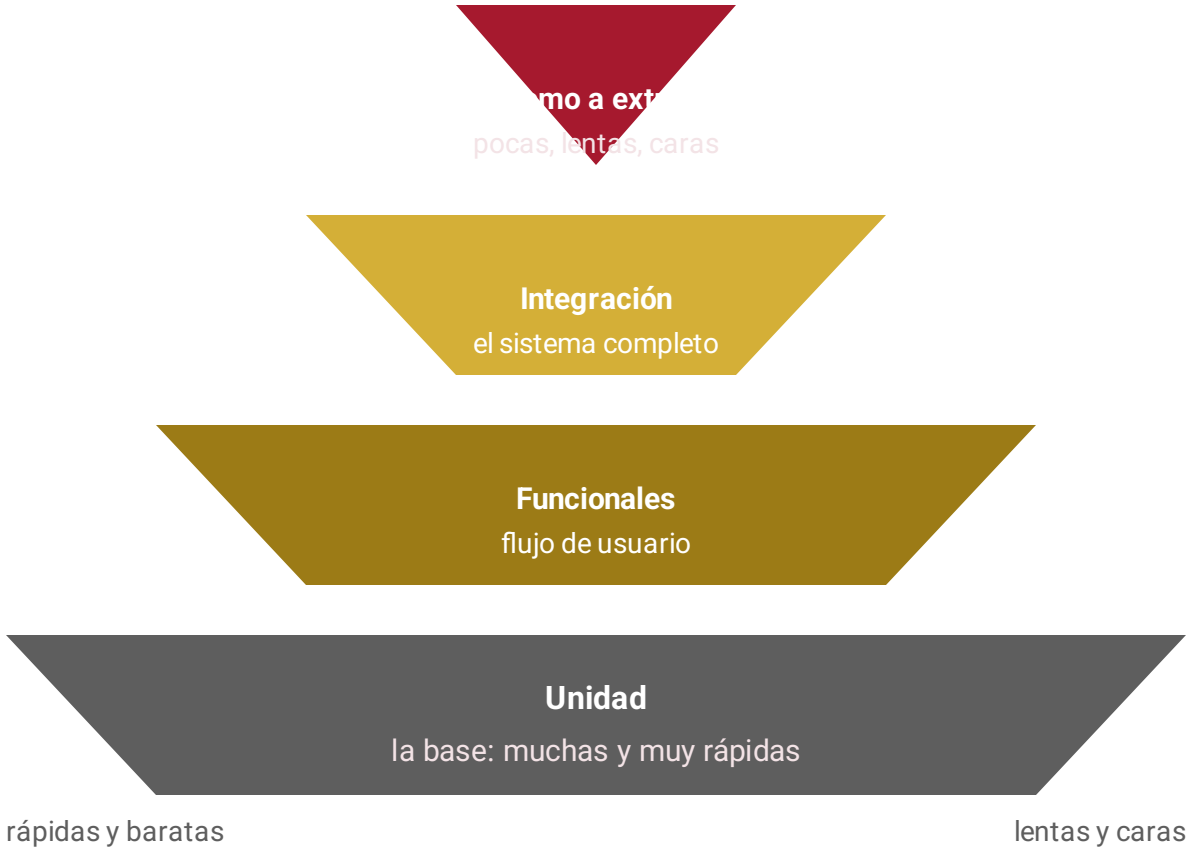
¿qué es lo que aprobaste?

Lo que pasó las pruebas fue el binario A, pero en producción corre el binario C. Si alguien pregunta qué se aprobó, la respuesta honesta es: otra cosa.

Basado en: Humble y Farley, Continuous Delivery, capítulo 10 [3] · Diagrama: elaboración propia

La pirámide: muchas baratas, pocas caras

Cada nivel entra en una puerta distinta, según cuánto tarda y cuánto cuesta correrlo.



Qué corre en cada puerta

En cada cambio
unidad + análisis estático

Al fusionar
integración + escaneo

Antes de publicar
extremo a extremo en staging

Después de publicar
comprobación funcional en el ambiente real, con alertas

Basado en: Humble y Farley, Continuous Delivery, capítulo 10 [3] · Diagrama: elaboración propia

Qué exige cada puerta para dejar pasar

Cuatro criterios explícitos: si no se puede comprobar, no se puede exigir.

1 Compila y pasa

Compilación sin errores y todas las pruebas de unidad en verde.

si no se cumple:
la rama ni se fusiona

2 Sin hallazgos graves

Escaneo de código, dependencias e imagen sin hallazgos graves.

si no se cumple:
no se publica nada

3 Artefacto verificable

Etiqueta fija, huella registrada y origen conocido de cada dependencia.

si no se cumple:
se congela el avance

4 Aprobación formal

Para el ambiente final, una persona distinta del autor aprueba el paso.

si no se cumple:
se espera, sin rogar



El criterio

Toda puerta exige un dato que el sistema puede medir solo. Si un criterio necesita interpretación humana en cada ejecución, no es una puerta: es una cola de espera.

Basado en: DORA, guía de métricas de entrega [1] · NIST SP 800-204C, seguridad en despliegue [5]

Cuatro escaneos que corren solos

Ninguno espera un clic: se ejecutan en la puerta que les corresponde y dejan evidencia.

Código
análisis estático

Patrones inseguros en el código fuente, antes de que alguien lo ejecute.

cada cambio

Dependencias
lista de materiales

Bibliotecas ajenas con vulnerabilidades o licencias que no permites usar.

al fusionar

Secretos
credenciales

Claves, tokens o contraseñas escritas dentro del repositorio o en la imagen.

cada cambio

Imagen
paquetes y sistema

Paquetes del sistema operativo y ajustes de configuración dentro del contenedor.

al empaquetar

Automatizado y trazable

Los cuatro corren sin pedir clic, guardan su hallazgo con la ejecución y la puerta lee ese registro. Es el mismo enfoque que las organizaciones publican como juego.

Etiqueta para los humanos, huella para la máquina

La etiqueta puede moverse; la huella, no. En producción se promueve la huella.



Lo que se aprobó

Etiqueta

nombre legible para las personas

Se escribe en el reporte y en el chat.
Puede volverse a apuntar a otra huella.

Lo que se despliega

Huella

resumen criptográfico del contenido

Es única y no cambia: si el contenido
cambia, cambia la huella. Así de simple.

Basado en: Fowler, Continuous Integration [4] · Diagrama: elaboración propia

Mismo camino, distinto contexto

Los ambientes se parecen en el recorrido y difieren en datos, escala y aprobación.



Basado en: GitHub, usar ambientes para el despliegue [10] · Diagrama: elaboración propia

La respuesta tiene que llegar en minutos

Si el resultado llega cuando el autor ya cambió de tarea, ya no informa: solo molesta.

un cambio, una corrida, una respuesta

la respuesta llega mientras el contexto sigue abierto



commit	compila	prueba	escanea	informa
minuto 0	minuto 1	minuto 3	minuto 5	minuto 6

Lo que sí cabe en minutos

Compilación, pruebas de unidad, análisis estático y escaneo de dependencias.

todo eso va en la primera puerta

Lo que corre después

Pruebas de extremo a extremo, despliegue y verificación en el ambiente real.

sin bloquear el siguiente cambio

Basado en: Humble y Farley, Continuous Delivery, capítulo 10 [3] · Tiempos ilustrativos, elaboración propia

ANTI-PATRÓN

Un solo paso gigante no es un pipeline

Cuando todo corre junto y al final, el fallo obliga a repetirlo todo.

Así se ve el error clásico

una tarea, catorce acciones

1 compilar 2 instalar dependencias

3 correr pruebas 4 generar reporte

5 escanear 6 aprobar manualmente

7 construir imagen 8 etiquetar

9 subir al registro 10 copiar al servidor

11 reconfigurar 12 reiniciar servicio

13 verificar a ojo 14 escribir el correo

sin puertas: si falla el paso 13, empieza de cero

sí →

lo mismo, en etapas cortas

- compilar y probar · respuesta inmediata
- escanear y empaquetar · evidencia propia
- publicar y desplegar · reversión lista
- verificar y avisar · informe automático

Cada bloque se puede repetir solo, y el fallo se queda donde ocurrió

seis acciones dentro de una sola tarea no hacen falta: haz cuatro tareas

Elaboración propia · caso CaribeMart (ficticio)

Cuatro datos que responden ¿qué está corriendo?

Si el pipeline guarda estas cuatro cosas, la auditoría se responde en segundos.

Quién lo cambió

commit

a3f9c21 · rama main

autor y mensaje

Qué pasó en la cadena

corrida

ejecución 842 · 5 etapas

resultado por etapa

Qué se publicó

artefacto

huella sha256…9f31

etiqueta y registro

Dónde terminó

despliegue

staging · 07:42 · ok

ambiente, hora y estado



La pregunta del lunes por la mañana

¿qué versión está en producción y quién la aprobó?

respuesta: commit a3f9c21 · ejecución 842 · huella …9f31 · staging, 07:42

Basado en: Humble y Farley, Continuous Delivery, capítulo 10 [3] · Diagrama: elaboración propia

CaribeMart: una publicación, siete minutos

El mismo cambio del hilo conductor, recorriendo la cadena completa.

```
# corrida 842 · rama main · carrito-servicio
build     compilar y probar          ok    48 s
test      214 pruebas de unidad       ok    1 min
scan      código y dependencias       ok    32 s
image     construir y etiquetar       ok    1 min
publish   registro · huella 9f31      ok    9 s
approve   autor distinto del que       ok    2 min
          subió el cambio
deploy    staging con la misma        ok    40 s
          huella
verify    funcional y alertas         ok    31 s

8 pasos · 1 aprobación formal · evidencia en un solo lugar
```

Lo que no pasó: nadie pidió un servidor, nadie copió un archivo, nadie escribió un correo.

Lo que sí pasó: una aprobación formal de dos minutos, con la huella a la vista.

Si algo hubiera fallado: la corrida se detiene en esa etapa y el equipo recibe el detalle.

Caso del curso: CaribeMart (proyecto de aula, ficticio) · Tiempos ilustrativos, elaboración propia

Dónde se pierde el tiempo de verdad

Cuatro esperas típicas y el dato que las delata en el registro.

La cola de aprobación

se siente como lentitud del equipo

La etapa técnica terminó y la corrida espera a una persona. El reloj sigue corriendo.

se delata: tiempo en estado esperando

El ambiente compartido

dos corridas, un solo destino

Las ejecuciones se turnan por un ambiente que no es efímero ni se crea y se destruye.

se delata: espera antes de ejecutar

La imagen pesada

todo el sistema operativo, cada vez

Cada corrida transmite y verifica una capa gigante que casi nunca cambió.

se delata: tiempo de transferencia

La prueba que nadie acelera

lenta por costumbre, no por diseño

La suite más larga corre en la primera puerta y frena cada cambio del equipo.

se delata: duración por etapa

ninguna de estas cuatro espera se ve en una cifra promedio

Basado en: DORA, guía de métricas de entrega [1] · Diagrama: elaboración propia

Cinco reglas para revisar tu cadena

Si alguna no se cumple, ahí está el siguiente trabajo de automatización.

1. **Se compila una vez**
   el artefacto que pasó las pruebas es el que llega a producción, sin reconstruir

2. **Lo repetible se automatiza**
   lo que se hace a mano dos veces es un candidato a la cadena, no una costumbre

3. **Cada paso deja evidencia**
   resultado por etapa, huella del artefacto y quién aprobó, todo en el mismo lugar

4. **Retroceder es tan fácil como avanzar**
   si volver atrás requiere un procedimiento, la cadena todavía no está terminada

5. **Los secretos viven fuera del código**
   nada de credenciales en el repositorio ni en la imagen: eso lo revisa la puerta

RESUMEN

Lo que dejó la parte dos

Cinco ideas para llevarte antes de abrir un motor de automatización.

1 · Etapas y puertas

Compilar, probar, escanear, publicar, desplegar y verificar, con una puerta automática entre cada una.

2 · Un solo artefacto

Se compila una vez y la huella viaja a todos los ambientes. Lo que se aprueba es lo que corre.

3 · Seguridad adentro

Código, dependencias, secretos e imagen se escanean solos, en la puerta que les toca.

4 · Minutos, no días

La respuesta rápida va primero; las pruebas lentas corren sin frenar el siguiente cambio.



5 · La cadena se escribe en un archivo

Todo lo anterior vive en un texto versionado con el código: la definición de las etapas, sus puertas y quién tiene permiso para tocar cada paso.

Siguiente: un motor de automatización

parte 3 · flujos, secretos y ambientes

Elaboración propia · síntesis de la parte 2

PARTE 3 DE 4

03

GitHub Actions: workflows, secretos y ambientes

Un motor de automatización como ejemplo, con su anatomía, sus ejecutores y dos casos colombianos.

PARTE 1
Métricas de DORA: qué mide un buen pipeline

PARTE 2
Etapas del pipeline: construir, probar, escanear, publicar

PARTE 3
GitHub Actions: workflows, secretos y ambientes

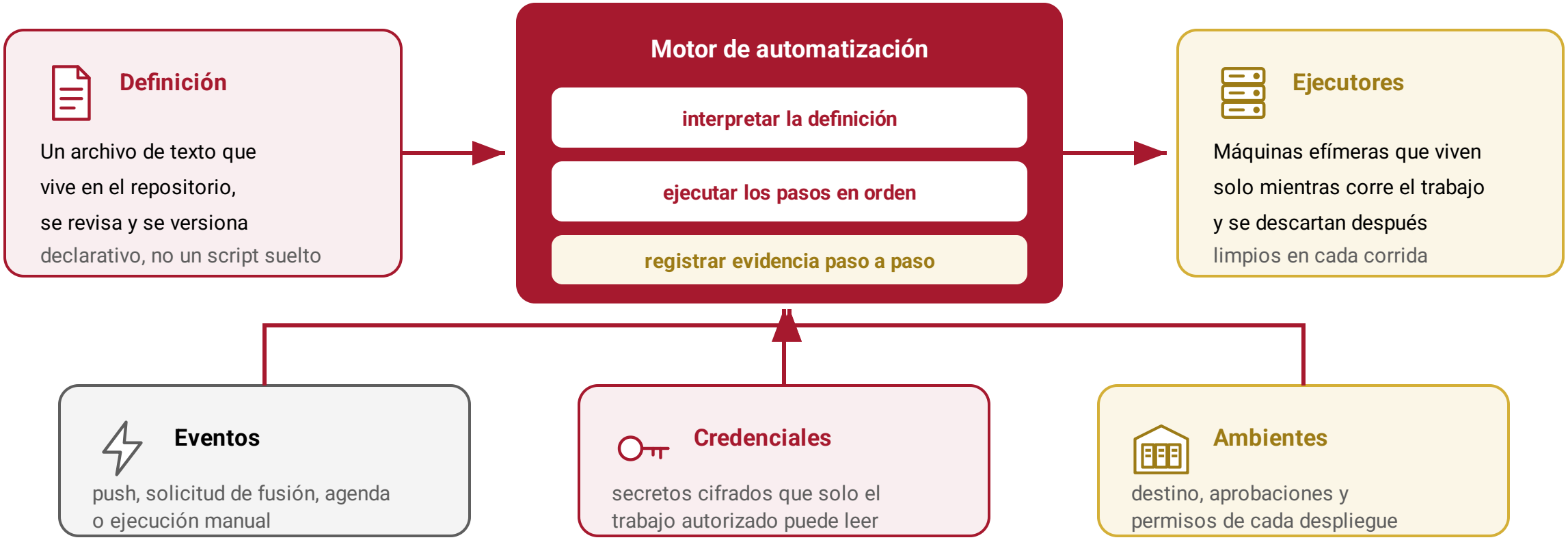
PARTE 4
Despliegue: push, GitOps y estrategias de publicación

CUC UNIVERSIDAD DE LA COSTA

CONCEPTO

Un motor de automatización tiene cinco piezas

Primero el concepto genérico; después veremos un producto concreto como ejemplo.



Basado en: GitHub, entender GitHub Actions [6] · Diagrama: elaboración propia

ANATOMÍA

Flujo, trabajos y pasos: tres niveles anidados

Los trabajos pueden correr en paralelo; los pasos de un trabajo, siempre en orden.

flujo · definición en el repositorio

disparadores

push a main · solicitud de fusión · agenda de lunes · manual

trabajo A · verificar

- **paso 1** instalar dependencias
  si falla, se detiene el trabajo
- **paso 2** correr las pruebas
  cada paso publica su resultado
- **paso 3** escanear el código
  el resumen queda en la corrida

trabajo B · empaquetar

- **paso 1** construir la imagen
  etiqueta con el commit
- **paso 2** escanear el contenedor
  paquetes y configuración
- **paso 3** subir al registro
  huella guardada como evidencia

Cómo se leen juntos

- Los trabajos A y B pueden correr al mismo tiempo, cada uno en su ejecutor.
- Dentro de un trabajo, los pasos van en orden: si uno falla, el resto no corre.
- El disparador decide cuándo arranca el flujo; el resto del tiempo nadie espera nada.

cada corrida queda guardada
con su resultado por paso

Basado en: GitHub, entender GitHub Actions [6] · Diagrama: elaboración propia

EJEMPLO

La misma cadena, escrita en un archivo

Estructura mínima de una definición: nombre, disparadores y trabajos con sus pasos.

```
# definición en el repositorio
nombre: verificar-cambios

disparadores:
  push:    ramas: [ main ]
  solicitud: ramas: [ main ]
  agenda:  - cron: "0 12 * * 1"

trabajos:
  verificar:
    ejecutor: ubuntu-latest
    pasos:
      - instalar dependencias
      - correr pruebas unitarias
      - escanear código
  empaquetar:
    necesita: verificar
    pasos:
      - construir imagen
      - escanear contenedor
      - subir al registro
```

El archivo vive en el repo: se revisa en una solicitud de fusión como cualquier código.

Los pasos son declarativos: qué hacer, no cómo instalar la máquina que lo hace.

Dependencias entre trabajos: empaquetar espera a que verificar pase sus puertas.

Dónde corre: administrado o propio

La misma definición apunta a una u otra opción sin cambiar los pasos.

Administrado por el proveedor

máquinas listas, se alquilan por uso

Arranca en segundos y no hay que parchear nada: el sistema operativo y las herramientas ya están.

ideal para pruebas rápidas
entorno distinto al de ejecución real

Propio, bajo tu control

máquinas que ya tienes o alquilas

Útil cuando el trabajo necesita una red privada, un equipo concreto o requisitos de cumplimiento.

ideal para despliegues internos
toca mantenerlo y escalarlo tú

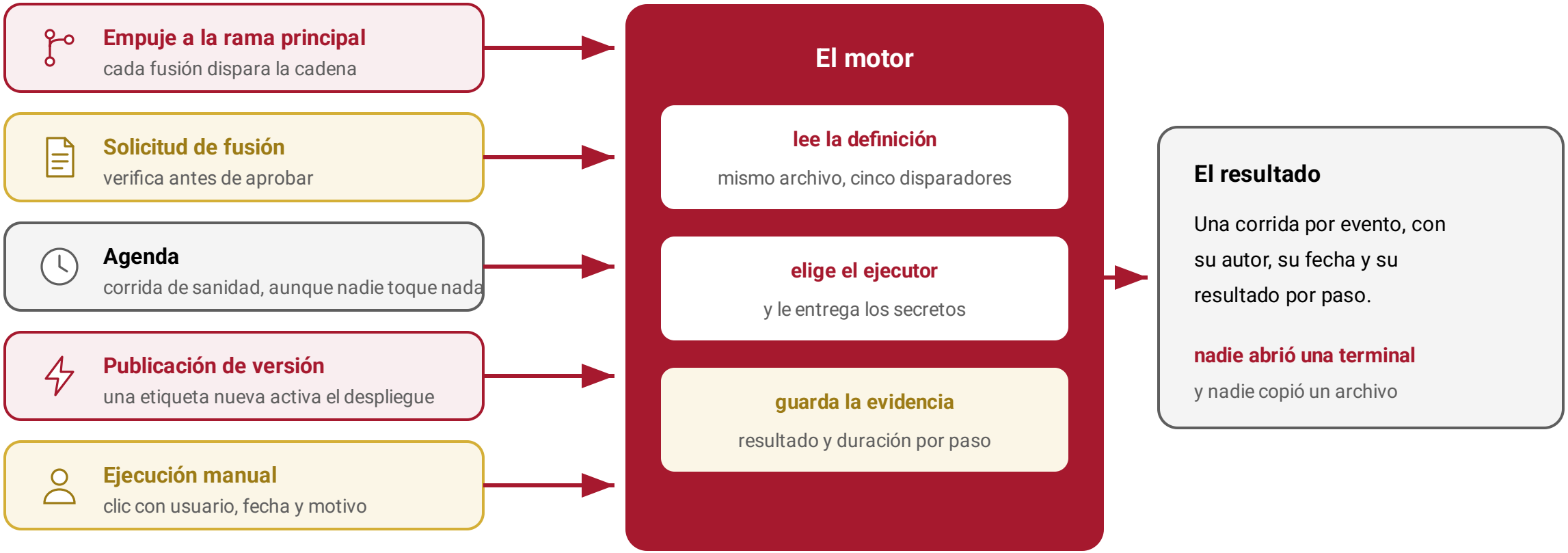
Lo que no cambia

El archivo de definición es el mismo en los dos casos: cambia la línea que declara dónde corre, no los pasos ni las puertas que ya construimos.

Basado en: GitHub, ejecutores [11] · Diagrama: elaboración propia

Qué enciende el flujo

Cinco formas de arrancar la misma cadena, sin abrir una terminal.



Basado en: GitHub, entender GitHub Actions [6] · Diagrama: elaboración propia

Los secretos viven en el servidor, no en el archivo

Una credencial en el repositorio es un incidente, no un atajo.

Así sí

Servidor de secretos
cifrado, con registro de quién lo cambió

El paso lo recibe como variable
solo durante esa corrida

En el registro de la corrida
aparece enmascarado, jamás en texto plano

Así no

Credencial en el archivo
dentro de la definición o de la imagen

Queda en el historial
y en cada copia del repositorio

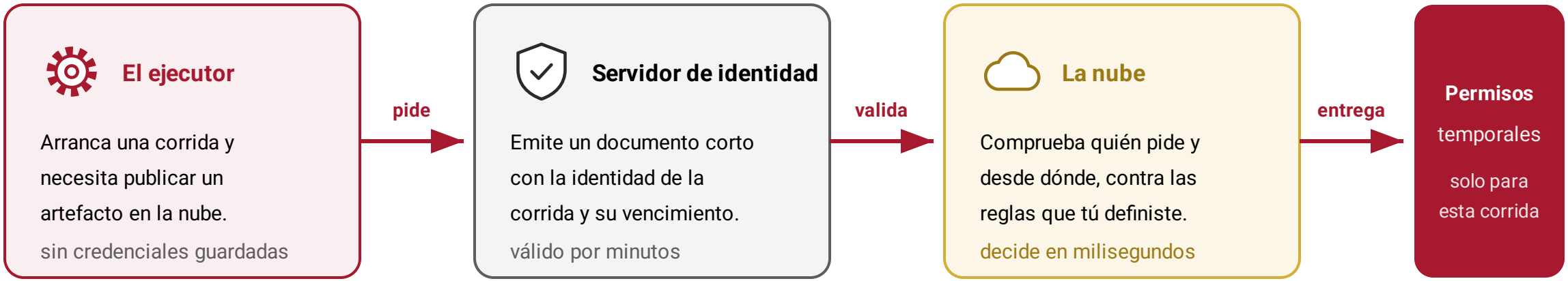
La puerta lo detecta
el escaneo de secretos detiene la cadena

La regla
Si un secreto se copió una sola vez en el repositorio, ya se considera comprometido: se rota y se retira.

Basado en: GitHub, usar secretos en los flujos [7] · Diagrama: elaboración propia

Acceso sin secretos: la identidad es la credencial

El ejecutor pide credenciales temporales presentando quién es y de dónde viene.



Por qué importa

Antes había que guardar una llave de larga duración en el servidor de secretos y renovarla a mano. Ahora la identidad de la corrida hace de credencial y el permiso muere solo cuando termina el trabajo.

Reclamos: lo que la nube revisa antes de conceder

No se confía en el nombre: se confía en el documento y en sus campos verificables.

El documento de la corrida

campos firmados y legibles

audiencia
para quién vale este documento

sujeto
repositorio, rama y ambiente exactos

vencimiento
minutos, no días

Si alguno no coincide con lo esperado, la nube responde negativa y el trabajo se detiene.

La regla que tú escribes

solo desde el repositorio de la aplicación
y solo desde la rama principal

solo hacia el ambiente de producción
con la aprobación ya registrada

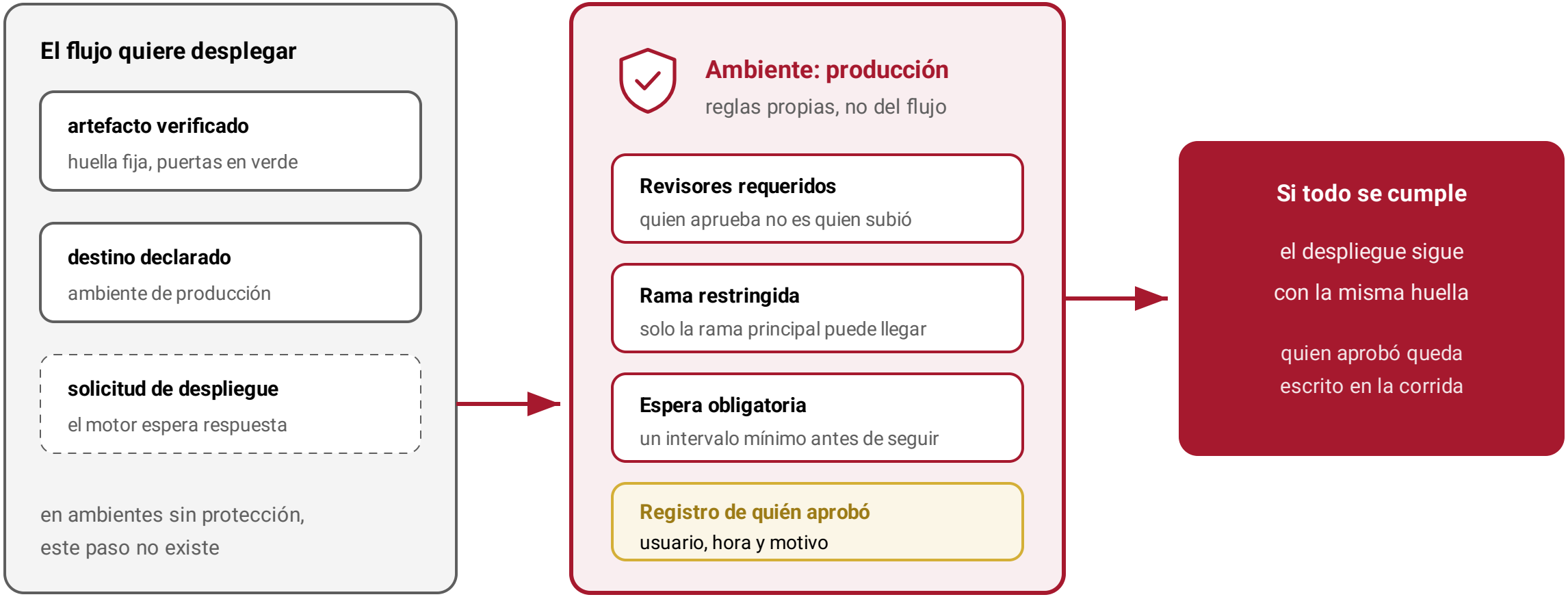
solo durante los minutos que dura la corrida
después, el permiso deja de existir

Resultado
ningún archivo de credenciales que olvidar

Basado en: GitHub, identidad federada entre flujos y la nube [8] · Diagrama: elaboración propia

Protecciones del ambiente: la puerta con persona

El despliegue final se abre solo si se cumplen las reglas del ambiente.



Basado en: GitHub, usar ambientes para el despliegue [10] · Diagrama: elaboración propia

El flujo declara solo lo que necesita

Mínimo privilegio: todo lo no declarado queda denegado.

Sin declaración

el token llega más lejos

Un paso que solo iba a leer archivos puede acabar con permisos de escritura en todo el repositorio.

nadie pidió eso

Declarado en el archivo

contenido: leer
el código, nada más

paquetes: escribir
solo para subir el artefacto

identidad: otorgar
para pedir permisos en la nube

todo lo demás: denegado

Resultado

Si un paso intenta algo fuera de lo declarado, falla y se ve en la corrida.

El daño posible queda acotado a lo que el trabajo realmente pidió.

Mínimo privilegio

Se declara lo necesario para el paso y nada más. La identidad temporal de la parte anterior hace lo mismo, pero al revés: el permiso nace corto y muere corto.

CASO REAL

Un banco de la región ya opera así

Repositorio público de un banco colombiano: cinco decisiones que reconocerás.

Lo que se observa en su cadena

- Ejecutor administrado, listo y efímero
- Ambiente de publicación declarado en el flujo
- Identidad federada en lugar de llave fija
- Acciones de terceros fijadas por versión
- Análisis de código en cada corrida

todo publicado en repositorio abierto

Qué demuestra

Las cinco decisiones de la parte dos y de esta parte no son teoría de aula: se usan en producción en la región y son auditables.

Cómo se lee en clase

Como ejemplo, no como receta: las piezas son del patrón, la herramienta elegida por ellos es solo una decisión de equipo.

Fuente: Bancolombia, repositorio devsecops-engine-tools [12] · Lectura de su configuración de automatización

Cuando el regulador publica lineamientos

La Superintendencia de Industria y Comercio describe la misma cadena en un documento formal.

El documento
lineamientos técnicos

Lineamientos de implementación DevOps
Superintendencia de Industria y Comercio · OTI

Versión 1.5 · 30 de noviembre de 2020
con control de cambios publicado

Su historial dice algo importante: las versiones siguientes añadieron el lineamiento de pipeline de integración continua y el de despliegue continuo.

Las mismas piezas, en lenguaje de gobierno

Puertas de calidad
una sección propia de controles antes de avanzar

Codificación y construcción
repositorio, ramas, versiones y verificación estática

Pipeline de integración continua
lineamiento dedicado, añadido en una versión posterior

Despliegue continuo
también con lineamiento propio en el documento

Si tu cadena habla este idioma, la conversación con cumplimiento es corta

Fuente: Superintendencia de Industria y Comercio, Lineamientos de implementación DevOps, versión 1.5, 2020 [13]

MINI-RETO

Pausa el video: ¿dónde va cada credencial?

PAUSA Y RESPONDE

CaribeMart necesita dos cosas en su flujo: subir la imagen al registro y generar el informe de dependencias. Hoy hay una llave de administrador escrita en el archivo de definición. ¿Qué harías con esa llave y cómo conseguirías los dos permisos sin guardar ninguno?

Pista: separa **secretos**, **identidad federada** y **permisos mínimos**; son tres mecanismos distintos.

Escribe la solución en tres líneas: dónde vive la credencial, quién la emite y qué alcance tiene.



llave de administrador

escrita en el archivo de definición



permisos que faltan

registro de imágenes e informe de deps



tres líneas

tu propuesta

RESUMEN

Lo que dejó la parte tres

Cinco ideas para llevarte antes de ver el despliegue.

1 · Concepto primero

Definición, motor, ejecutores, eventos y ambientes: el vocabulario de cualquier herramienta.

2 · Tres niveles

Flujo, trabajos y pasos: paralelo entre trabajos, en orden dentro de cada uno, con evidencia propia.

3 · Secretos y llaves

El secreto vive en el servidor; la identidad federada lo reemplaza y el permiso nace corto.

4 · Puertas con reglas

Ambiente, revisores, rama y permisos viven en el destino, no en el archivo del flujo.



5 · El ejemplo es un ejemplo

Las piezas son del patrón y las usan organizaciones de la región; la marca concreta solo sirve para mostrar dónde se aprieta cada botón del camino.

Siguiente: cómo se publica de verdad

parte 4 · empuje, GitOps y estrategias

PARTE 4 DE 4

04

Despliegue: push, GitOps y estrategias de publicación

Quién mueve el cambio al clúster, los principios de OpenGitOps, azul y verde, y canario.

PARTE 1
Métricas de DORA: qué mide un buen pipeline

PARTE 2
Etapas del pipeline: construir, probar, escanear, publicar

PARTE 3
GitHub Actions: workflows, secretos y ambientes

PARTE 4
Despliegue: push, GitOps y estrategias de publicación

CUC UNIVERSIDAD DE LA COSTA

Quién mueve el cambio: tú, o el destino

Empujar y pedir resuelven la misma tarea con filosofías distintas de confianza.

Empujar

la cadena lleva el cambio hasta el destino

Cadena de entrega
tiene la credencial

envía →

Ambiente
recibe y aplica

Rápido y simple: quien construye también publica. La llave vive del lado del automatismo.

el destino no decide: aplica lo que llega

Pedir

el destino busca y aplica el cambio

Repositorio
estado deseado

compara →

Ambiente
observa y ajusta

Más lento de armar y más robusto: cada destino mira lo que debe estar corriendo.

el destino tiene credenciales solo para leer

La pregunta de la semana

No cuál es mejor, sino quién queda con la llave de cada ambiente y qué pasa si se pierde.

Basado en: OpenGitOps, principios de GitOps [2] · Diagrama: elaboración propia

Cuatro principios que definen GitOps

Son la definición abierta del modelo de operación declarativa.

1 · Declarativo

Se describe el resultado que se quiere, no la lista de instrucciones para lograrlo.

estado deseado escrito, no comandos sueltos

2 · Versionado e inmutable

Ese estado vive en un repositorio, con historial, revisiones y reversión por confirmación.

quién cambió qué y cuándo, en un solo lugar

3 · Obtenido automáticamente

El sistema toma ese estado por su cuenta, sin que nadie copie archivos a mano.

el mismo camino para todos los ambientes

4 · Reconciliado continuamente

Lo que corre se compara sin descanso con lo que debería correr; si se separa, se corrige.

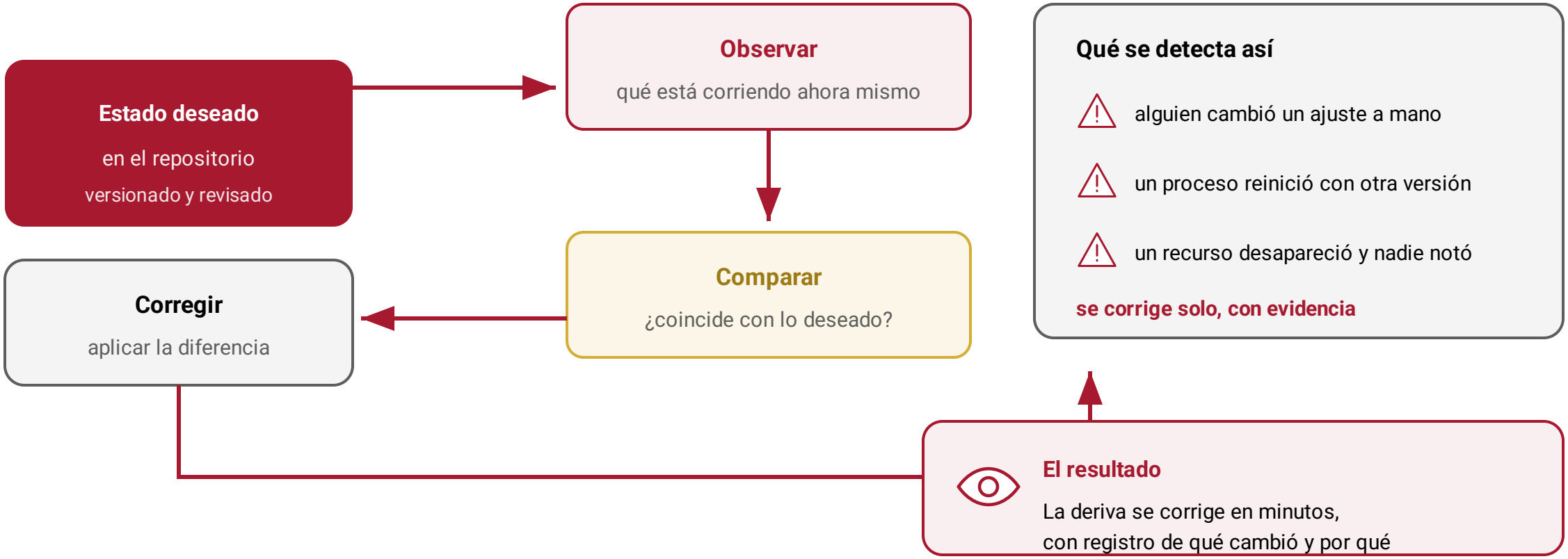
la deriva no espera a que alguien la note

los cuatro juntos: sin uno de ellos, el resto deja de ser sostenible

Fuente: OpenGitOps, principios de GitOps [2] · Diagrama: elaboración propia

Reconciliar: observar, comparar y corregir

El corazón del modelo: nadie espera a que alguien se dé cuenta de la deriva.



Fuente: OpenGitOps, principios de GitOps [2] · Diagrama: elaboración propia

CRITERIOS

Cuándo empujar y cuándo pedir

No es una religión: la elección depende de urgencia, número de ambientes y gobierno.

Conviene empujar cuando…

- el cambio es urgente y reversible
- hay pocos ambientes y todos se parecen
- quien despliega es quien construyó
- la operación es de un solo equipo

menos piezas en movimiento, más cerca del código

Conviene pedir cuando…

- hay muchos ambientes que mantener iguales
- operación y desarrollo son equipos distintos
- importa quién cambió qué, con historial
- el destino debe resistir cambios manuales

más gobierno, menos credenciales repartidas



La regla práctica

Muchos equipos usan los dos: empujan a pruebas y pedir en producción, donde el gobierno pesa.

Basado en: OpenGitOps, principios de GitOps [2] · Humble y Farley, Continuous Delivery [3]

El riesgo del empuje: quién queda con la llave

Una credencial de larga duración en la cadena convierte una fuga en un incidente mayor.

El escenario frágil

Llave de escritura en la cadena
guardada como secreto, vigente por años

La cadena toca el destino directamente
si se filtra, se publica lo que sea

El alcance del daño lo decide la llave,
no la intención de quien la usa.

El escenario sólido

Identidad de la corrida
sin secreto almacenado, permiso por minutos

Alcance declarado por destino
lo que la regla permite, ni un poco más

Si algo se filtra, se filtra una identidad
corta, acotada y revocable.

Lo que se lleva la clase

La pregunta no es si la llave se puede filtrar, es qué tan grande queda el daño cuando eso pasa. La identidad temporal y el alcance declarado reducen esa superficie.

Basado en: GitHub, identidad federada y secretos [8] · GitHub, secretos en los flujos [7]

Por qué no se publica todo de golpe

El tamaño del daño posible, no el código, decide cómo se publica.

De golpe

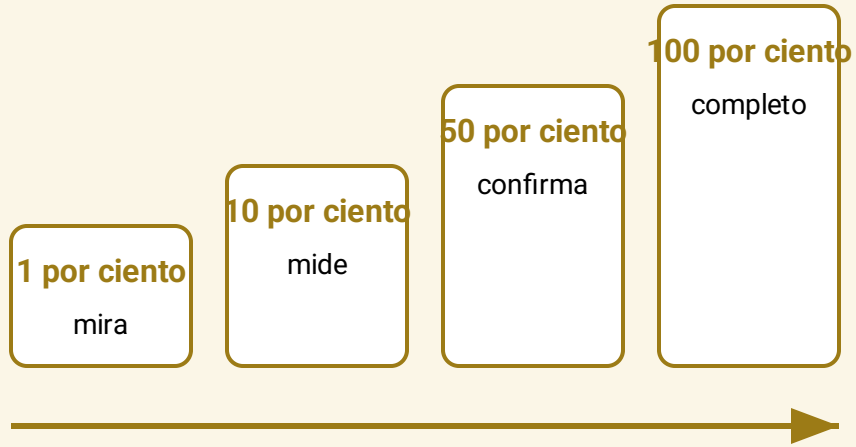
una sola decisión, todo el tráfico a la vez



Si algo anda mal, no hay público que quede afuera: todos reciben el problema a la vez.

Por etapas

cada paso se gana con evidencia



La idea de la semana

Publicar es una serie de decisiones pequeñas, no un interruptor.

Basado en: Humble y Farley, Continuous Delivery [3] · Diagrama: elaboración propia

Azul y verde: dos ambientes y un conmutador

La versión nueva espera llena de tráfico ficticio antes de recibir a nadie real.



Los cuatro pasos

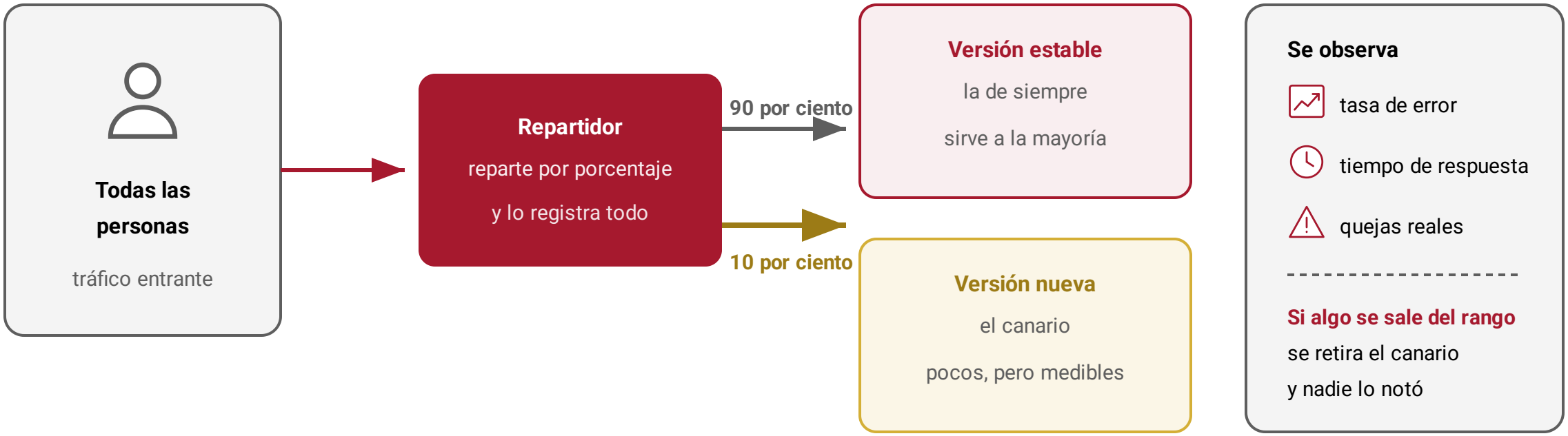
1 · publicar verde · 2 · verificar con pruebas y métricas · 3 · conmutar el tráfico · 4 · azul queda en espera

Si algo falla en el paso 3, el conmutador regresa a azul en segundos

Basado en: Humble y Farley, Continuous Delivery [3] · Diagrama: elaboración propia

Canario: primero una parte mínima del público

Se expone un porcentaje pequeño y se decide con datos, no con impresiones.



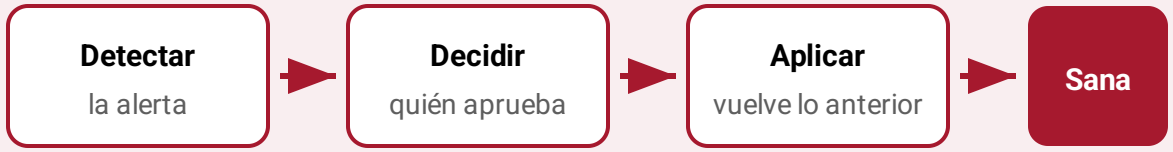
La diferencia con azul y verde
no se cambia todo el tráfico de una: se compra información antes de comprometer a todos.

Basado en: Humble y Farley, Continuous Delivery [3] · Diagrama: elaboración propia

El botón rojo: revertir es una tarea planificada

Poder volver atrás no es suerte de último minuto; es algo que se diseña y se ensaya.

La ruta cuando algo sale mal



Nadie improvisa: el camino está escrito, probado y con una persona designada.

Para que el botón sirva

- la versión anterior sigue disponible
- los datos aceptan volver atrás
- el regreso se ensayó antes, no el día D

sin los tres, revertir es solo una intención



el botón solo existe si detrás hay un proceso

Después se aprende

Qué pasó, por qué no se vio antes y qué puerta faltó en la cadena.

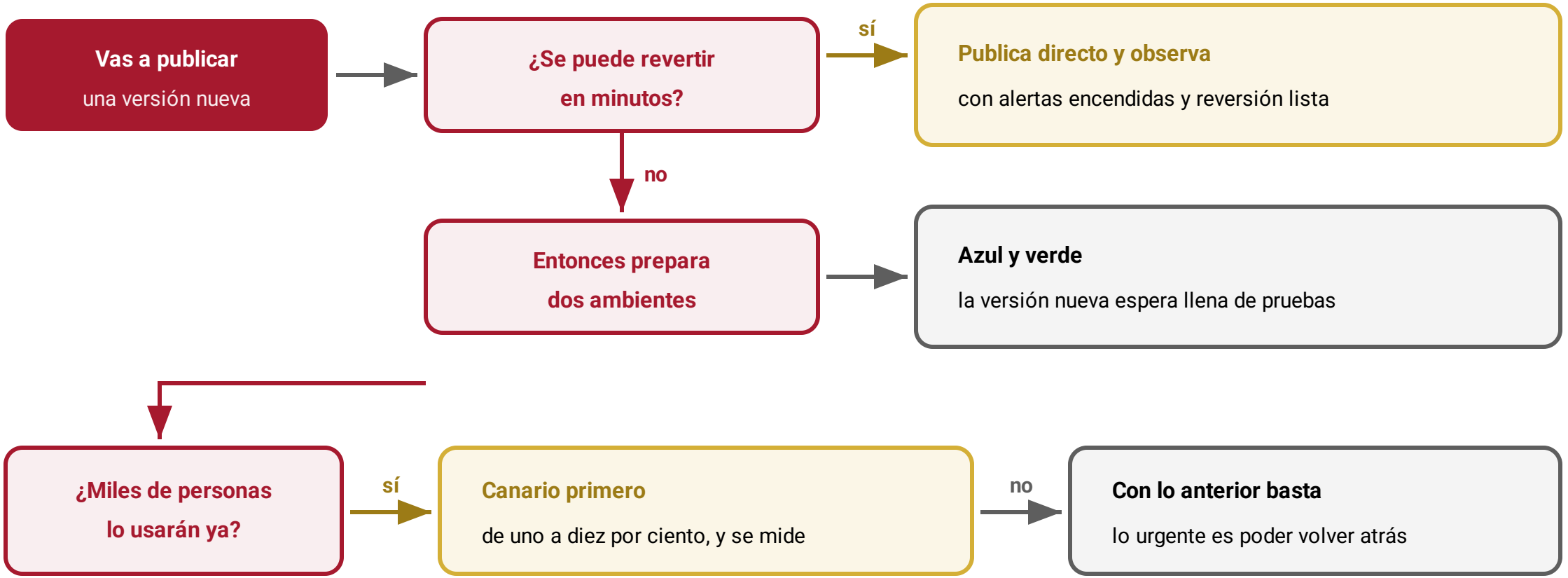
la reversión compra tiempo, no conocimiento

Fuente: Humble y Farley, Continuous Delivery [3] · Diagrama: elaboración propia

DECISIÓN

Qué estrategia elijo para esta publicación

El árbol se recorre por el riesgo del cambio, no por lo que esté de moda.



La estrategia se elige por el riesgo y el daño posible, no por la moda del equipo

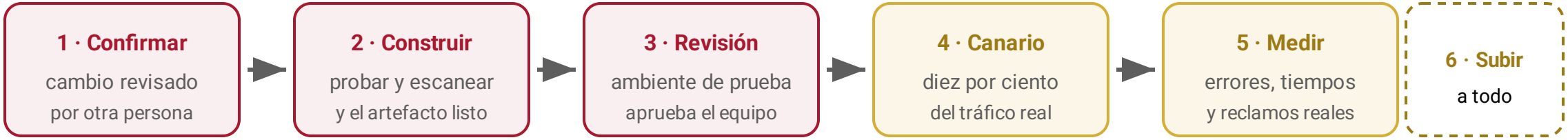
Basado en: Humble y Farley, Continuous Delivery [3] · OpenGitOps [2]

Cómo publica CaribeMart una semana normal

Los dos modelos conviven: se empuja a pruebas y se pide en producción.

La ruta de CaribeMart

se confirma un cambio, la cadena hace el trabajo pesado y la decisión de publicar sigue siendo de la gente



Lo que se empuja

Las pruebas, el escaneo y el artefacto: todo corre solo, sin esperar a que alguien lo mande a hacer.

velocidad donde no hay riesgo

Lo que se pide

La decisión de tocar producción: alguien aprueba, las métricas acompañan y recién entonces se sube.

criterio donde sí hay riesgo

Caso de clase CaribeMart (hilo conductor del curso) · Diagrama: elaboración propia

PREGUNTA ABIERTA

Si nadie estuviera mirando, ¿qué pasaría esta noche?

No hay respuesta única: sirve para discutir qué tan frágil es el despliegue de tu entorno.

Si esta noche la cadena publicara sin que nadie estuviera mirando, ¿qué tan tranquilo te quedarías?

¿Hay reversión?

Si algo sale mal, ¿en cuántos minutos vuelve lo anterior?

y quién es la persona que decide

¿Hay puertas?

Si las pruebas fallan, ¿el cambio llega igual a producción?

y si alguien apaga la puerta a mano

¿Quién tiene la llave?

Si la credencial se filtra, ¿qué tan grande es el daño posible?

y cuánto tardarías en revocarla

Tres minutos en parejas, después compartimos dos respuestas con la clase

La semana en cinco ideas

Lo que debe quedarte aunque se te olvide el resto de la clase.

Cinco ideas para llevarte

1. Integración continua es una práctica, no una herramienta: compilar, probar y detectar en cada cambio.
2. La cadena gana confianza con puertas: si una falla, no se sigue y el cambio vuelve a quien lo hizo.
3. GitHub Actions se mueve con cuatro piezas: flujos, ejecutores, disparadores y ambientes protegidos.
4. Empujar y pedir responden a la misma pregunta: quién queda con la llave y qué tan grande es el daño.
5. Se publica por etapas y con reversión ensayada: primero uno a diez por ciento, y lo anterior sigue disponible.

PUENTE

Qué practicas antes de la semana 11

Cierra la Unidad 2 con cuatro tareas concretas y deja listo el repaso.

Cuatro tareas para consolidar

- Escribe un flujo que corra al confirmar un cambio
- Añade una puerta que detenga el despliegue al fallar
- Guarda un secreto y limita los permisos del flujo
- Decide, por escrito, la estrategia de tu publicación

y prepara la reversión: ¿qué volvería atrás y en cuántos minutos?

La próxima semana

Repaso Unidad 2

Recorremos de nuevo las piezas de la unidad: redes, patrones, contenedores y esta cadena de entrega, con preguntas abiertas.

prepara tus dudas de la unidad

Cierre de la Unidad 2

Con esta semana cierras el ciclo de CaribeMart: del cambio al despliegue, con métricas y reversión. Nos vemos en el repaso.

Elaboración propia · Calendario del curso: semana 11, Repaso Unidad 2

Fuentes citadas

1. DORA (Google Cloud), «DORA's software delivery performance metrics», guía actualizada el 5 de enero de 2026 — https://dora.dev/guides/dora-metrics/
2. OpenGitOps (CNCF), «GitOps Principles v1.0.0» — https://opengitops.dev/
3. Humble, J. y Farley, D., Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation (Addison-Wesley, 2010), capítulo 10 — https://www.informit.com/store/continuous-delivery-reliable-software-releases-through-9780321601919
4. Fowler, M., «Continuous Integration», martinfowler.com (2006, actualizado el 18 de enero de 2024) — https://martinfowler.com/articles/continuousIntegration.html
5. NIST SP 800-204C, Implementation of DevSecOps for a Microservices-based Application with Service Mesh — https://csrc.nist.gov/pubs/sp/800/204/c/final
6. GitHub Docs, «Understanding GitHub Actions» — https://docs.github.com/en/actions/learn-github-actions/understanding-github-actions
7. GitHub Docs, «Using secrets in GitHub Actions» — https://docs.github.com/en/actions/security-for-github-actions/security-guides/using-secrets-in-github-actions
8. GitHub Docs, «OpenID Connect» (conceptos de federación y token de corta vida) — https://docs.github.com/en/actions/concepts/security/openid-connect
9. GitHub Docs, «Configuring OpenID Connect in cloud providers» — https://docs.github.com/en/actions/how-tos/secure-your-work/security-harden-deployments/oidc-in-cloud-providers
10. GitHub Docs, «Using environments for deployment» (ambientes y reglas de protección) — https://docs.github.com/en/actions/deployment/targeting-different-environments/using-environments-for-deployment
11. GitHub Docs, «Runners» (ejecutores gestionados y propios) — https://docs.github.com/en/actions/reference/runners
12. Bancolombia, repositorio público devsecops-engine-tools con flujo de trabajo de GitHub Actions (workflows/build.yml) — https://github.com/bancolombia/devsecops-engine-tools
13. Superintendencia de Industria y Comercio, «Lineamientos de implementación DevOps», versión 1.5, 30 de noviembre de 2020 — https://sigi.sic.gov.co/SIGI/files/mod_documentos/anexos/304/Anexo%201%20Lineamientos%20GS03-P03.pdf